00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 02

Algoritma Analizi: İşlem Sayma, Asimptotik Gösterim ve Büyüme Hızları

RAM modeli · en iyi/ortalama/en kötü durum · O–Ω–Θ · uygulanabilirlik · oran testi · deneme bölmesi, kalbur, hızlı üs, Kadane

MODÜL I · ALGORİTMALARIN DİLİ

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** **Asimptotik Analiz**	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

Geçen haftadan: doğruluktan verimliliğe

- **H1'in üç aracı masada:** döngü değişmezi (başlatma–koruma–sonlanma), kâhinle karşılaştırmalı test, dikkatli ölçüm (ısınma, tekrar, ortanca).
- **AP-1 H9:** özyinelemeli Fibonacci'nin çağrı sayısı üstel büyüyordu → **işlem sayısının büyümesi**.
- **AP-1 H11:** satır satır ve sütun sütun dolaşma aynı işlem sayısı, farklı süre → **birim maliyetin sınırları**.
- **AP-1 H13:** `strlen` dizginin bütün karakterlerini tarar → **gizli maliyet**.
- **Mini-OBS 2 v2.0:** 10^5 kayıt 17,1 s, 10^6 kayıt için kestirim ≈ 1,6 saat; log–log eğim 2,52.

TAHMİN ET

H1’deki doğrusal arama, aranan değer dizide yoksa 10^6 elemanda kaç karşılaştırma yapar? $2 \cdot 10^6$ elemanda?

A 20 ve 21 (her adımda yarıya)

B 10^6 ve $2 \cdot 10^6$

C 10^6 ve $4 \cdot 10^6$

D Dizinin içeriğine bağlıdır

GERÇEK DÜNYADAN

2021

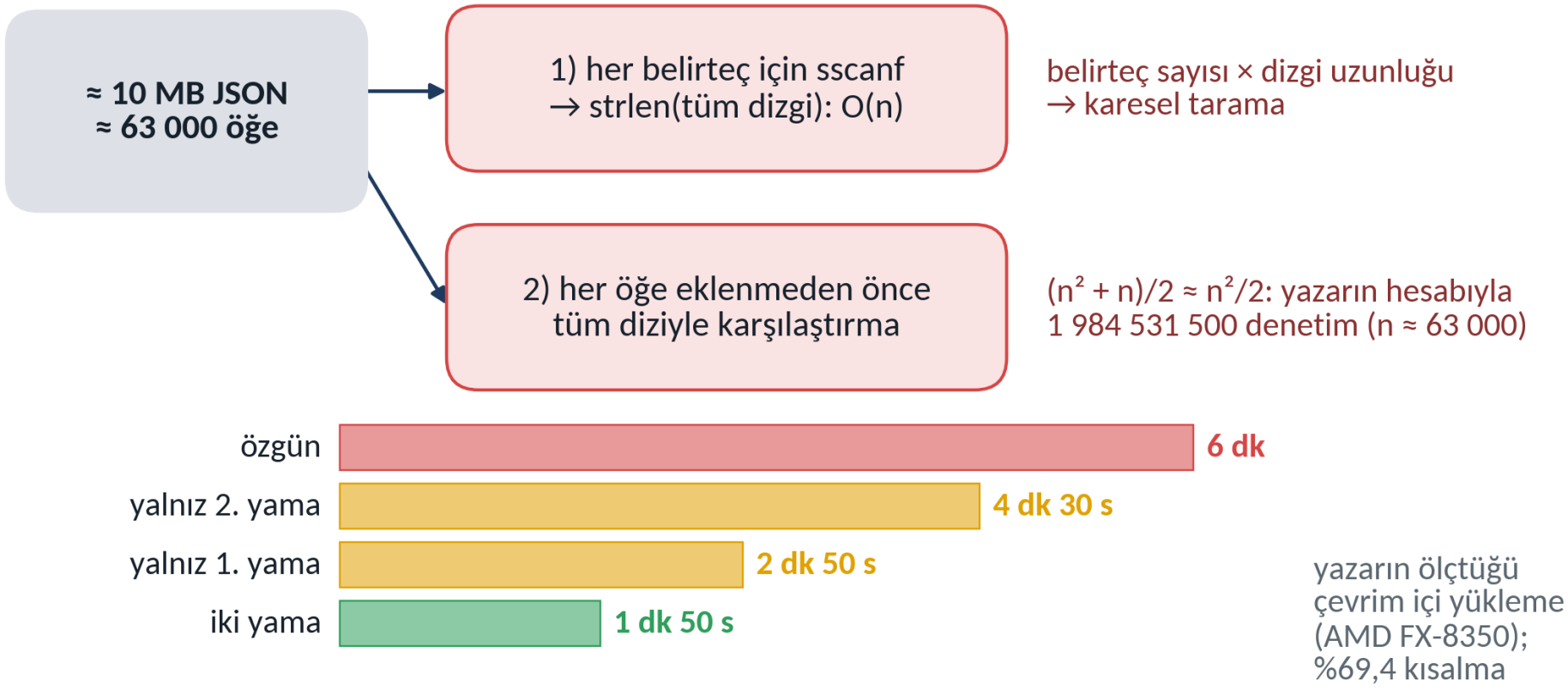
2021: GTA Online'ın altı dakikalık yükleme ekranı

- Bir oyuncu (takma adı **t0st**), çevrim içi modun kendi bilgisayarında (AMD FX-8350) ≈ **6 dakikada** yüklendiğini gördü; bunun ≈ 4 dakikasında tek bir çekirdek tam yükteydi, disk ve ağ boştaydı.
- ≈ **10 MB**'lık JSON ayrıştırılırken `sscanf` her çağrıda `strlen` ile uzunluğu baştan hesaplıyordu.
- ≈ **63 000** öğenin her biri eklenmeden önce dizideki bütün öğelerle karşılaştırılıyordu: 1 984 531 500 denetim (≈ $n^2/2$).
- İki düzeltme birlikte süreyi **1 dk 50 s**'ye indirdi (yazarın hesabıyla %69,4 kısalma); düzeltme 16 Mart 2021'de oyuna eklendi.

t0st (2021), How I cut GTA Online loading times by 70%, nee.lv

İki gizli karesel desen

GTA Online yüklemesindeki iki gizli karesel desen (t0st, 2021)



Her sscanf çağrısında kalan metnin uzunluğu baştan hesaplanır → toplam karesel; her öğe eklenmeden önce dizinin tamamı taranır → ≈ n²/2 denetim (t0st, 2021)

BÜYÜK SORU · TAHMİN ET

$n = 10^6$'da 1 saniye süren karesel bir program $n = 10^9$'da ne kadar sürer (aynı makine, aynı dil)?

A ≈ 1000 s (≈ 17 dakika)

B ≈ 1500 s (≈ 25 dakika)

C ≈ 10^6 s (≈ 11,6 gün)

D ≈ 10^9 s (≈ 32 yıl)

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 2.1	RAM modeli altında temel işlemleri sayarak bir algoritmanın çalışma süresini T(n) fonksiyonu olarak ifade eder.
ÖÇ 2.2	En iyi, en kötü ve ortalama durumu ayırt eder; basit algoritmalar için her birini belirler.
ÖÇ 2.3	O, Ω ve Θ gösterimlerini tanımlarına uygun kullanır; fonksiyonları büyüme hızlarına göre sıralar.
ÖÇ 2.4	Döngü yapılarından (ardışık, iç içe, bağımlı, yarıya inen) karmaşıklık sınıfını çıkarır ve oran testiyle deneysel olarak doğrular.
ÖÇ 2.5	Sayı kuramı algoritmalarını (deneme bölmesi, Eratosthenes kalburu, Öklid, hızlı üs alma) zaman ve bellek karmaşıklıkları açısından karşılaştırır.

İşlem sayma: RAM modeli ve temel işlem

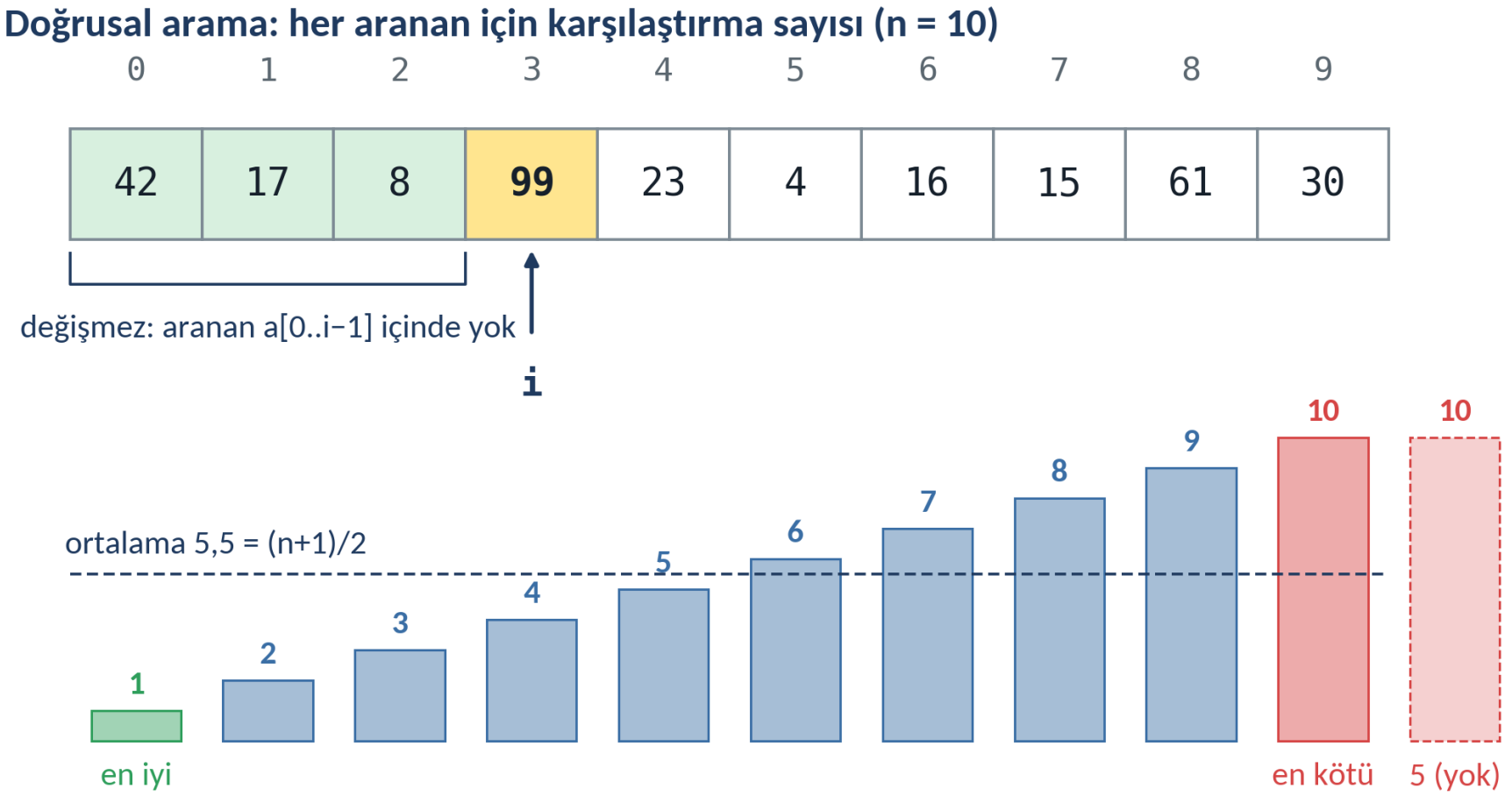
İşlem sayma: aynı döngü, iki maliyet modeli (Örnek 1)

adım (sözde kod)	RAM modeli	temel işlem: toplama
`toplam ← 0`	**1**	–
`i ← 0`	**1**	–
`i < n sınaması`	**n + 1**	–
`toplam ← toplam + a[i]`	**n**	**n**
`i ← i + 1`	**n**	–
toplam (kapalı biçim)	**T(n) = 3n + 3**	**T(n) = n**

İkisi de Θ(n): temel işlem seçimi yalnızca sabit çarpanı değiştirir (n = 0'da 3 adım, 0 toplama).

Aynı döngü: her ilkel adım sayılırsa T(n) = 3n + 3, yalnızca toplama sayılırsa T(n) = n (Örnek 1)

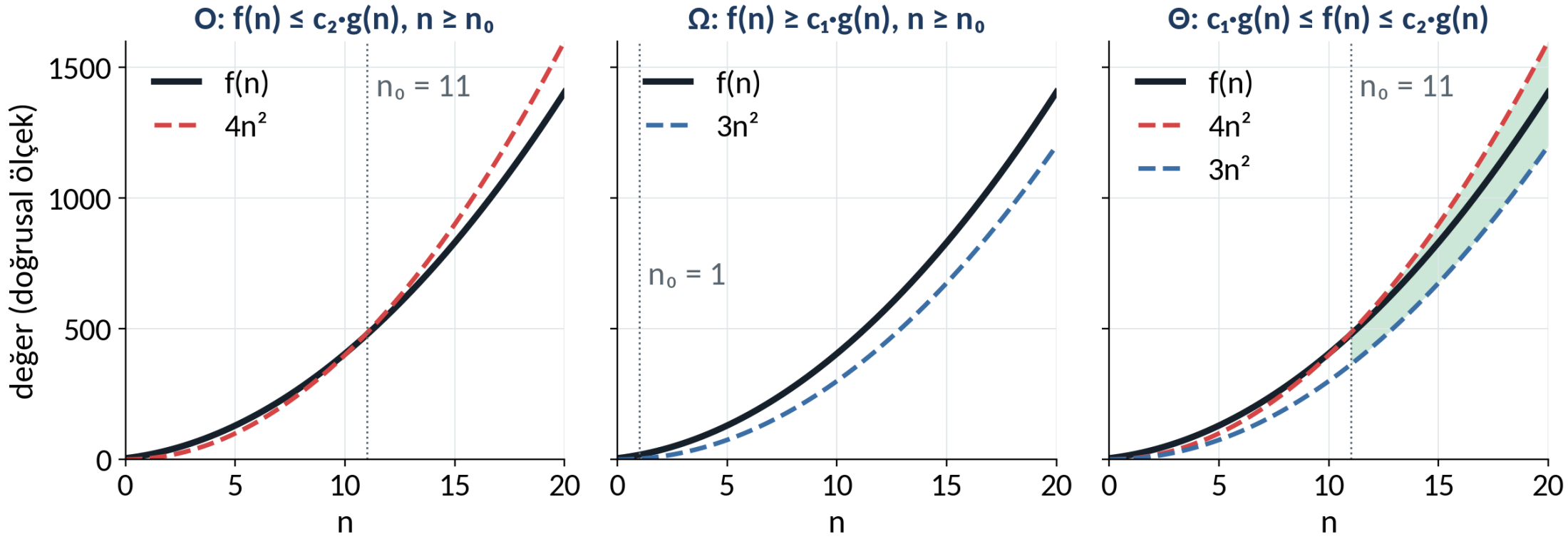
En iyi, en kötü ve ortalama durum



Doğrusal arama (n = 10): en iyi 1, en kötü 10 karşılaştırma; her konum eşit olasılıklıysa ortalama (n + 1)/2 = 5,5

O, Ω ve Θ: üst, alt ve sıkı sınır

$f(n) = 3n^2 + 10n + 5$, $g(n) = n^2$: $c_1 = 3$, $c_2 = 4$, $n_0 = 11$ → $f(n) = \Theta(n^2)$



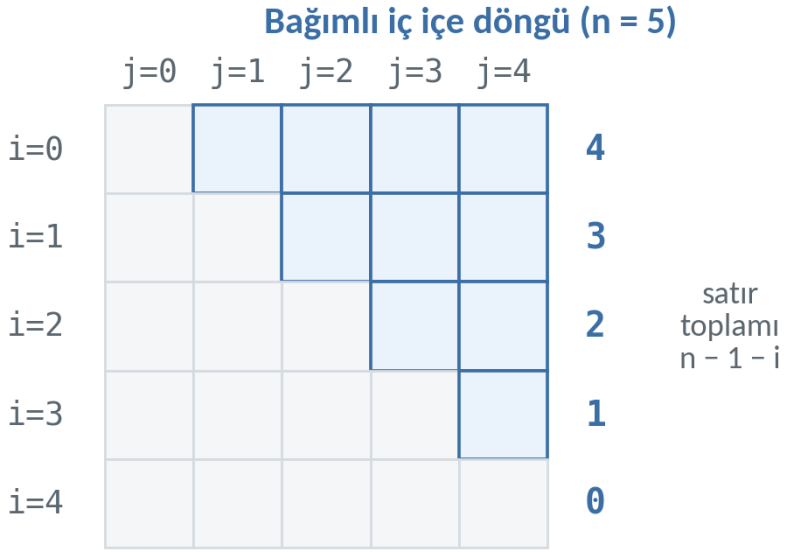
$4n^2 \geq 3n^2 + 10n + 5 \Leftrightarrow n^2 - 10n - 5 \geq 0 \Leftrightarrow n \geq 10{,}48 \rightarrow$ ilk tamsayı $n_0 = 11$ (kavram/H02_K03_sandvic_n0.py)

$f(n) = 3n^2 + 10n + 5$: $c_1 = 3$, $c_2 = 4$ ve $n \geq n_0 = 11$ için $3n^2 \leq f(n) \leq 4n^2 \Rightarrow f(n) = \Theta(n^2)$

Döngülerden karmaşıklık: altı desen

Desen	Yapı	Sayım	Sınıf
Ardışık	`for i < n; sonra for j < n`	n + n = 2n	Θ(n)
İç içe (bağımsız)	`for i < n: for j < n`	n · n	Θ(n²)
İç içe (bağımlı)	`for i < n: for j = i + 1 … n − 1`	n(n − 1)/2	Θ(n²)
Yarıya inen	`while n > 0: n /= 2`	⌊log₂ n⌋ + 1	Θ(log n)
Kare köke kadar	`for i = 2; i · i ≤ n`	⌊√n⌋ − 1	Θ(√n)
Dış katlanır, iç i'ye kadar	`for i = 1; i < n; i *= 2: for j < i`	1 + 2 + 4 + … < 2n	Θ(n)

Ardışık bloklar **toplanır**, iç içe bloklar **çarpılır**. Bağımlı iç içe döngü bir **üçgeni** doldurur: n(n − 1)/2 ≈ n²/2 hücre; sabit ½, sınıf yine Θ(n²). Son satır tuzaktır: iç içe olduğu hâlde Θ(n).



4 + 3 + 2 + 1 + 0 = 10 = n(n − 1)/2

bağımsız sürüm (j = 0'dan): 5 · 5 = 25 = n²

TAHMİN ET

`yariya_adim(n)` n = $2^{64} - 1$ için kaç kez döner?

H02_03_yariya_inen.c · satır 14–22

C

```c
int yariya_adim(unsigned long long n)
{
    int adim = 0;
    while (n > 0) {
        n /= 2;                    /* tamsayı bölmesi: aşağı yuvarlar */
        adim++;
    }
    return adim;
}
```

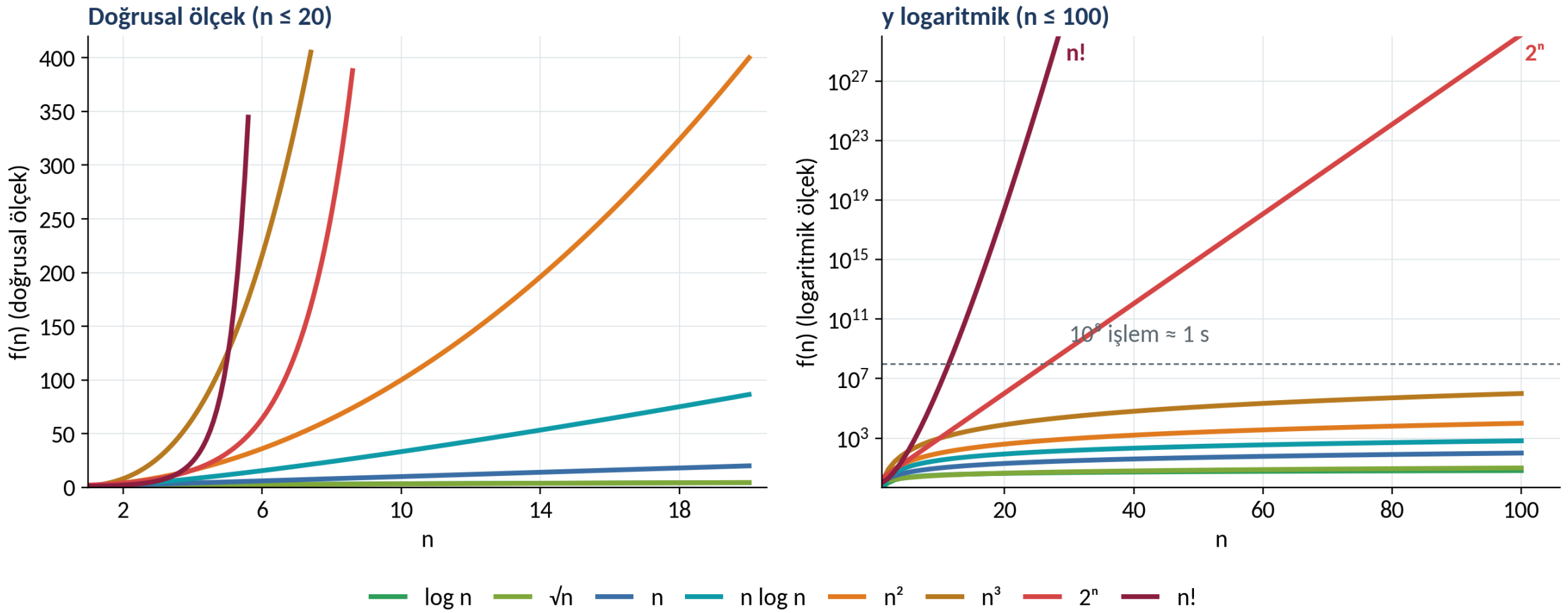
A 2^{63}

B 32

C 63

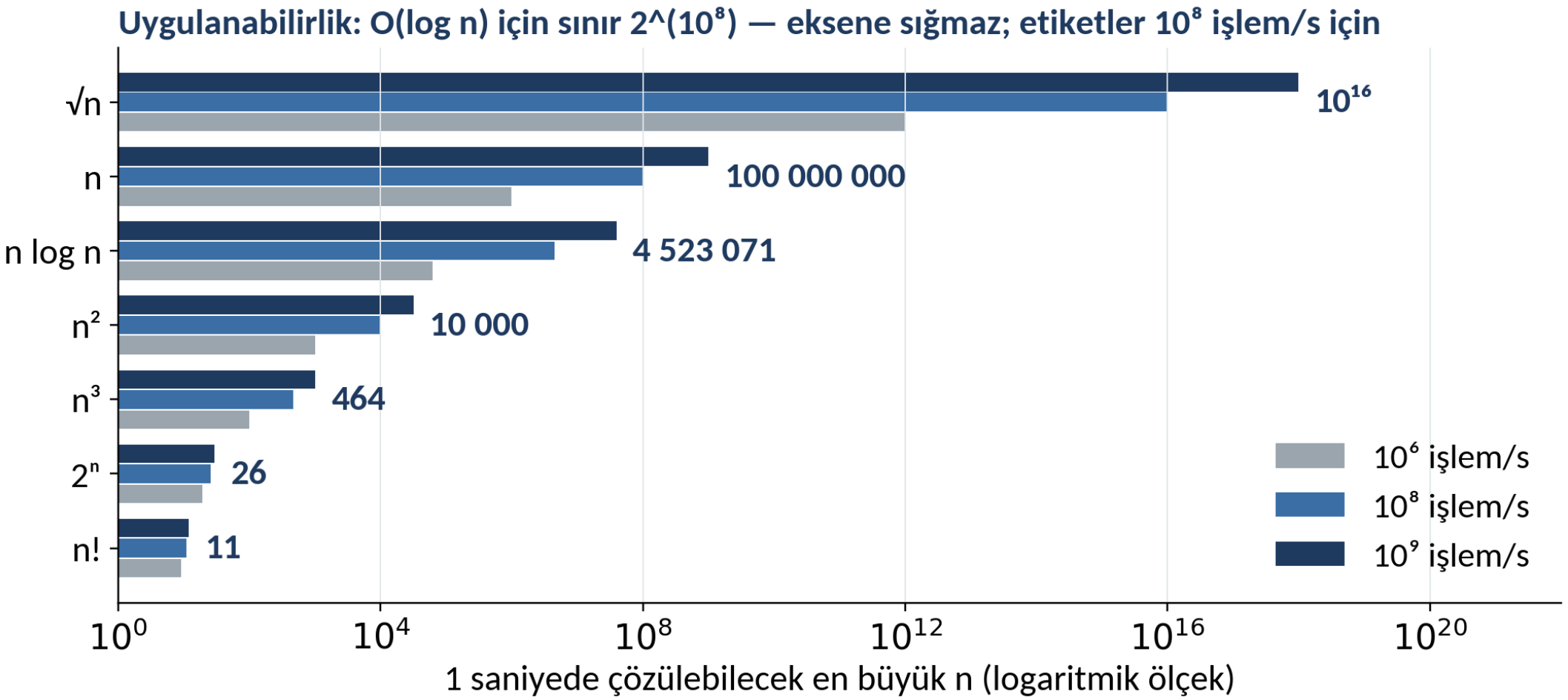
D 64

Büyüme sınıfları: doğrusal ve logaritmik ölçek



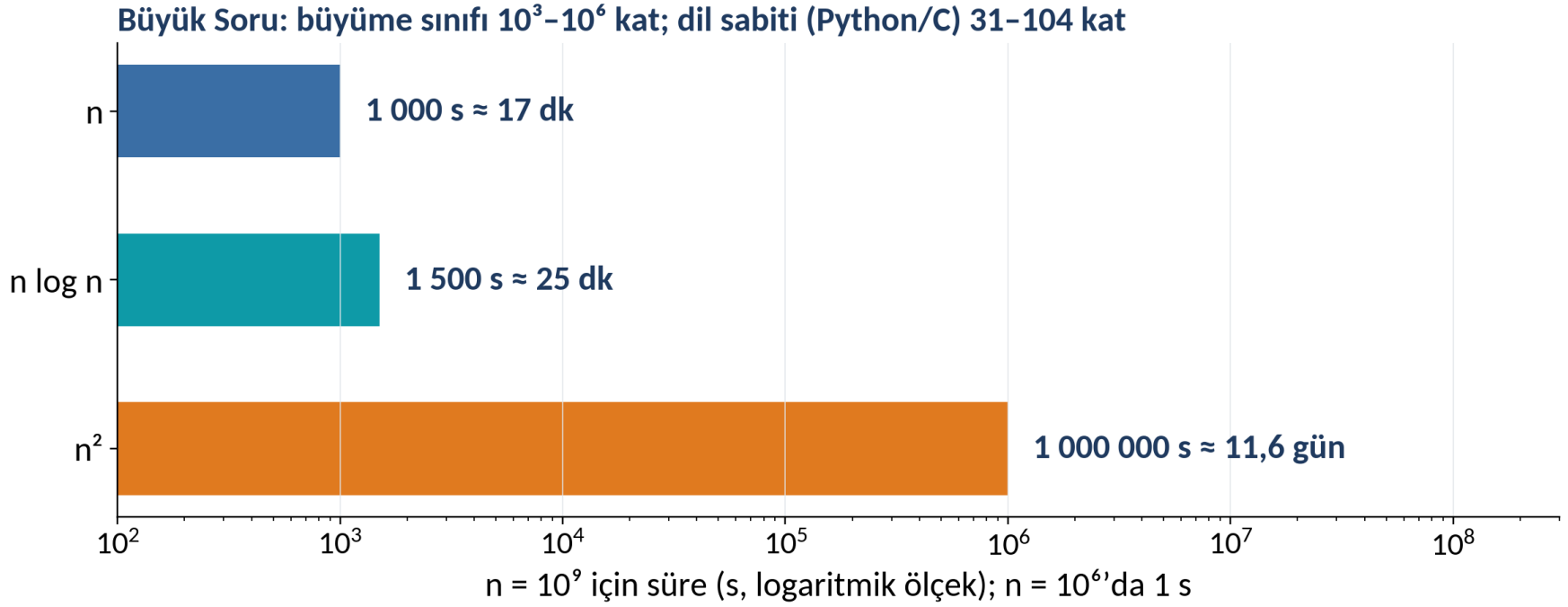
Solda doğrusal ölçek (n ≤ 20): n!, n^3 ve 2^n birkaç adımda grafikten çıkar; sağda y logaritmik ölçek (n ≤ 100); kesikli çizgi: 10^8 temel işlem ≈ 1 s

1 saniyede çözülebilecek en büyük n



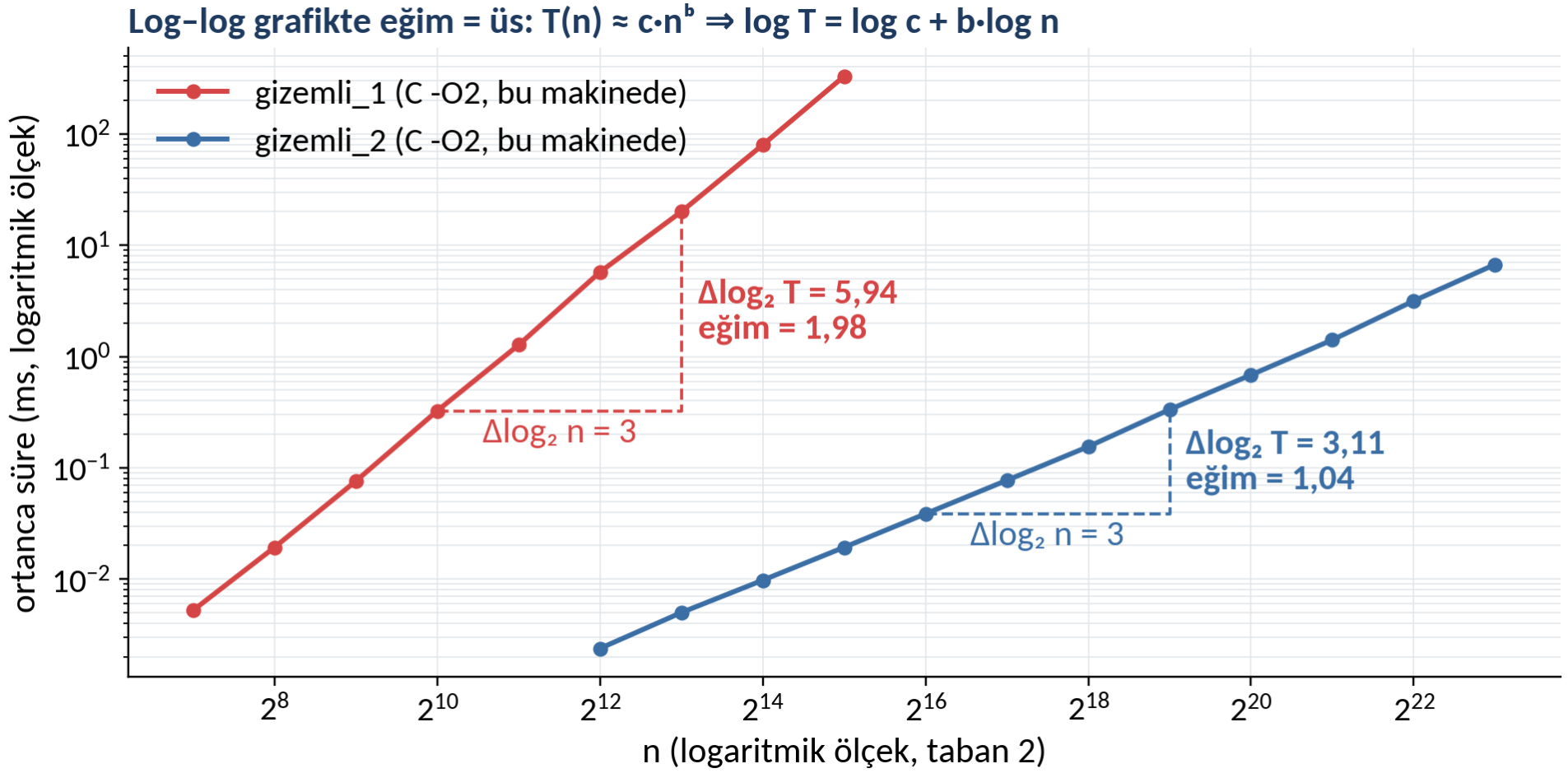
Örnek 6'nın yöntemiyle (x ekseni logaritmik ölçek; log n için sınır 2^(işlem sayısı), eksene sığmaz)

Büyük Soru'nun yanıtı: önce sınıf, sonra sabit



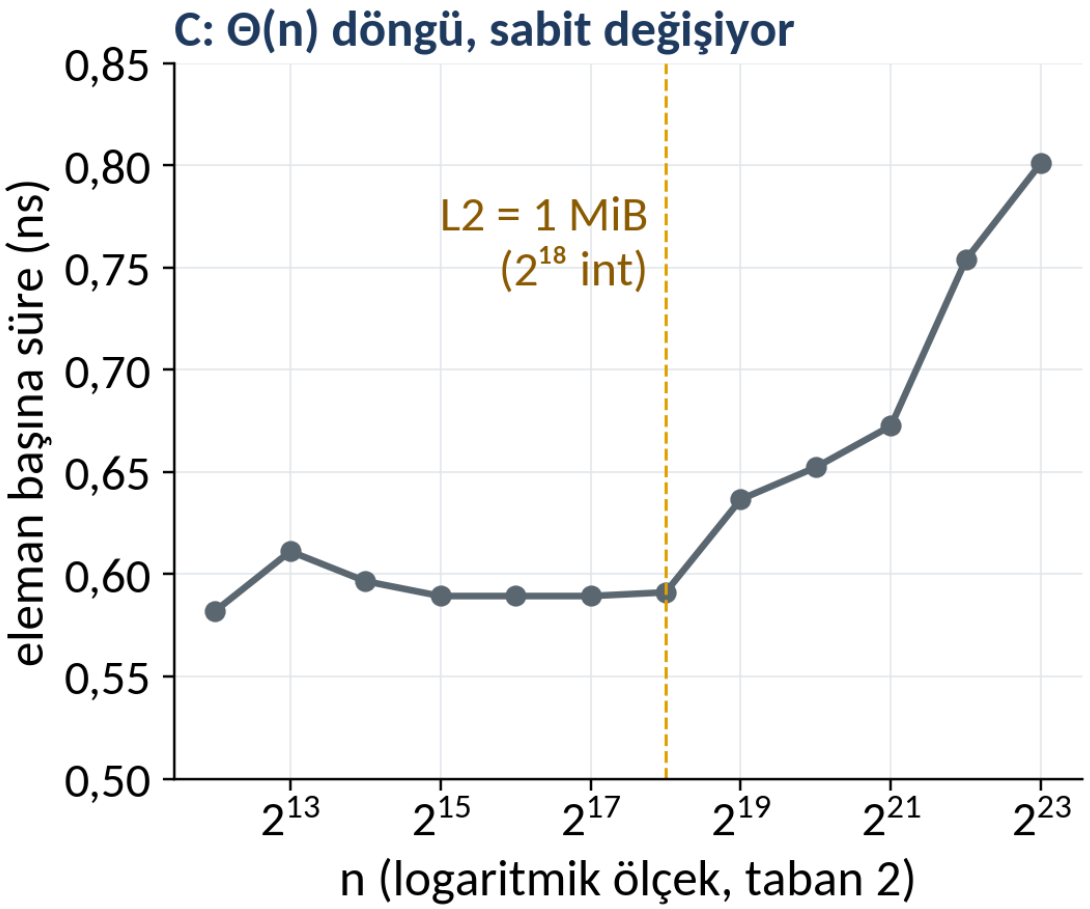
Önce sınıf: girdi 1000 kat büyüyünce süre doğrusal programda 10^3, karesel programda 10^6 kat artar ve bu fark n ile büyümeye devam eder. **Sonra sabit:** dil seçimi süreyi bir çarpanla değiştirir; bu makinede Python, aynı algoritmanın C sürümünden 31–104 kat yavaş ölçüldü (Haftanın Deneyi).

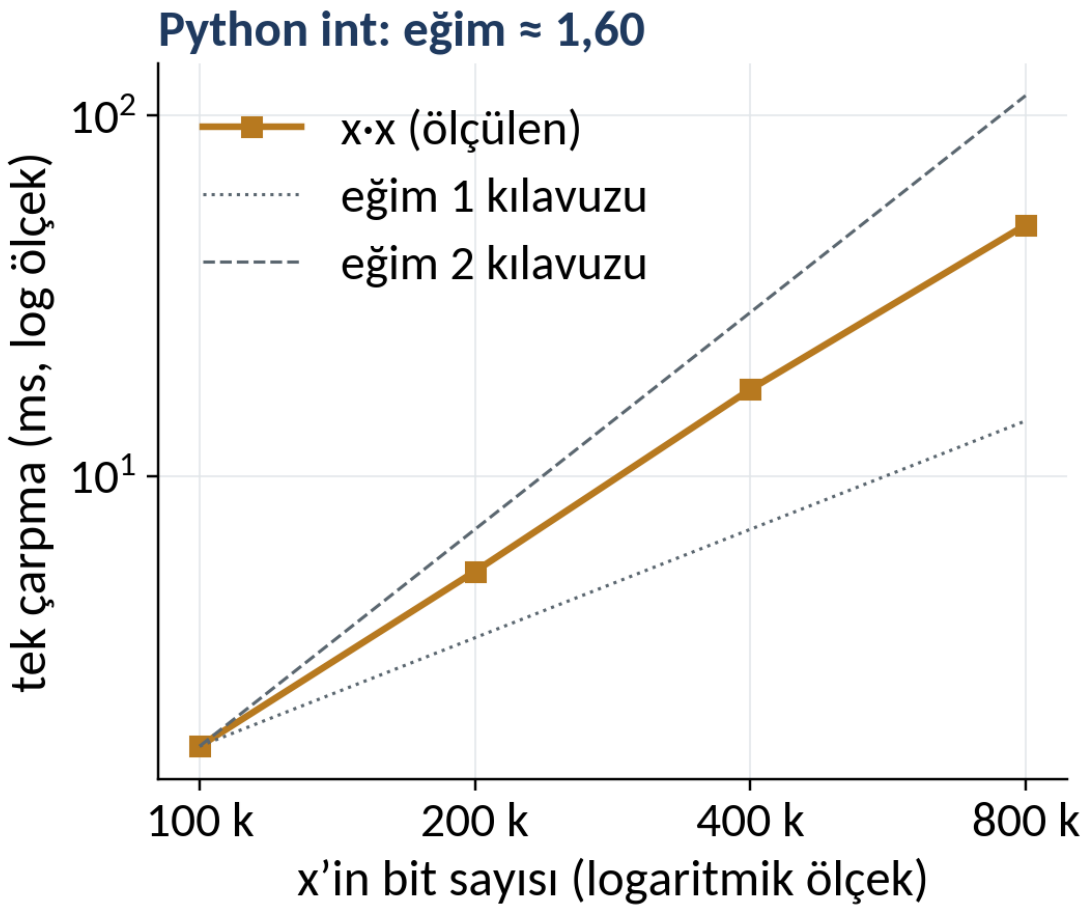
Oran testi ve log–log eğim: üssü ölçümden oku



$T(n) \approx c \cdot n^b$ ise $T(2n)/T(n) \approx 2^b$ (2 doğrusal, 4 karesel, 8 kübik) ve log–log grafikte eğim $b \approx \Delta\log_2 T / \Delta\log_2 n$

Kaputun Altında: birim maliyetin ve “sabit zamanın” sınırları





Sol: gizemli_2’de eleman başına süre L2 önbelleği aşılınca artar (eğim ≈ 1; dal tahmini de benzer). Sağ: Python’da x · x süresi bit sayısıyla büyür

DENEME BÖLMESİ · TAHMİN ET

`i * i <= n` sınırıyla n = 97 kaç bölmeyle asal ilan edilir? Saf yöntem (2 … n – 1) kaç bölme yapar?

A 8 ve 95

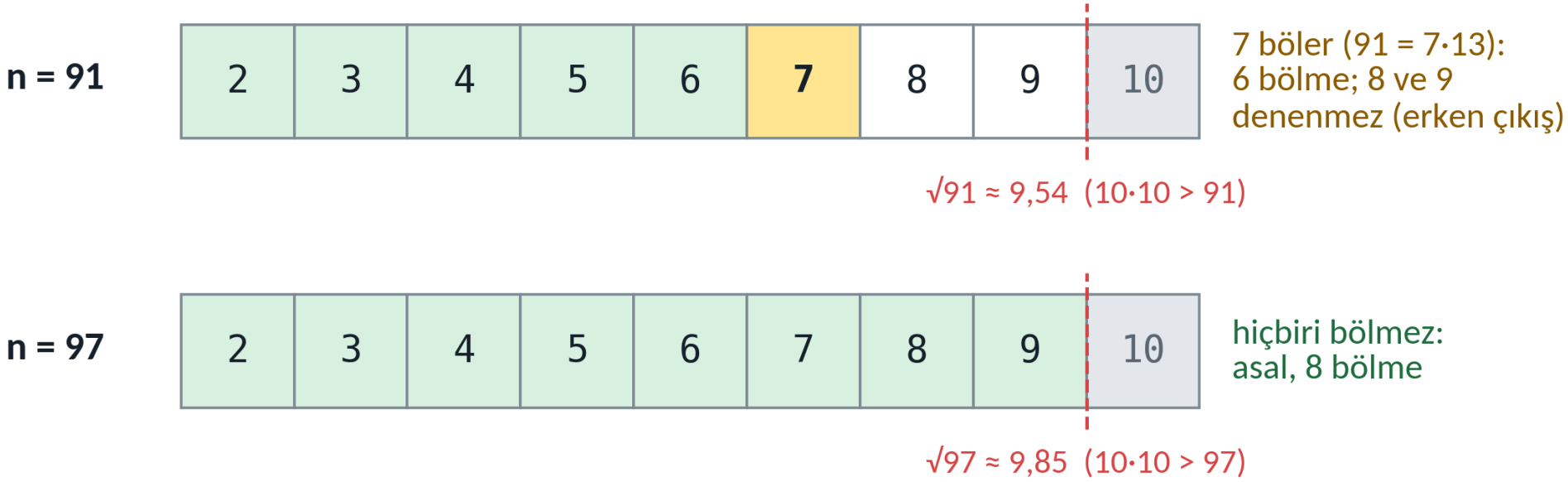
B 9 ve 96

C 48 ve 95

D 10 ve 97

Deneme bölmesi: adaylar yalnızca i · i ≤ n iken

Deneme bölmesi: adaylar yalnızca i·i ≤ n iken denenir



Neden √n yeter? n = d · e ise d ile e'nin ikisi birden √n'den büyük olamaz: 36 = 2·18 = 3·12 = 4·9 = 6·6.

Asal n'de ⌊√n⌋ − 1 bölme: n = 7919 için 87 (Örnek 8).

n = 91: 7 böler, döngü erken biter (bileşik); n = 97: 2 … 9 denenir, hiçbiri bölmez (asal, 8 bölme)

Sözde kod: deneme bölmesi

```
algoritma ASAL-MI(n)
    eğer n < 2 ise döndür YANLIŞ
    i ← 2
    i · i ≤ n iken tekrarla
        eğer n mod i = 0 ise döndür YANLIŞ
        i ← i + 1
    döndür DOĞRU
```

- **Değişmez:** döngü başında 2 ≤ d < i olan hiçbir d n'i bölmez.
- **Sonlanma:** i · i > n ise n bileşik olsaydı en küçük böleni d ≤ √n < i olurdu: çelişki.
- **Analiz:** en iyi (çift n) 1 bölme; en kötü (asal n) ⌊√n⌋ − 1 bölme: **O(√n)**; ek bellek O(1).
- Bit sayısı b ise √n ≈ 2^(b/2): girdi boyutuna göre üstel (Merak Kutusu).

KOD

C: asal_mi

H02_08_deneme_bolmesi.c · satır 22–35

C

```c
bool asal_mi(uint64_t n, uint64_t *bolme)
{
    *bolme = 0;
    if (n < 2) {
        return false;
    }
    for (uint64_t i = 2; i * i <= n; i++) {
        (*bolme)++;
        if (n % i == 0) {
            return false;   /* i böler */
        }
    }
    return true;
}
```

ÇIKTI (ÜÇ DİLDE AYNI; SEÇİLİ SATIRLAR)

```
                 n  asal  bölme(√n)  bölme(saf)
                97  evet          8          95
              7919  evet         87        7917
        2147483647  evet      46339           -
        1000000007  evet      31621           -
```

ALGORİTMA KİMLİK KARTI

Deneme bölmesiyle asallık *primality test by trial division*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet; adaylar 2 … ⌊√n⌋	$n \geq 0$; $n < 2^{63}$ (uint64_t'de i * i taşmasın)	**O(1)**	**—**	**O(√n)**	**O(1)**

Özellikler	Erken çıkış: ilk bölende durur (çift n'de 1 bölme). Asal n'de ⌊√n⌋ − 1 bölme (n = 10^9 + 7 için 31 621). Girdinin bit sayısına göre üsteldir: b bitlik n için ≈ 2^(b/2) bölme.
Değişmez / doğruluk	Döngü başında $2 \leq d < i$ olan hiçbir d, n'i bölmez
Kütüphane karşılığı	C: yok (elle) · C++: yok · Python: yok (`sympy.isprime` üçüncü taraf)
Ne zaman kullanılır	Tek tek sorgulanan, yaklaşık 10^{12}'ye kadar sayılar; ek bellek istenmiyorsa
Ne zaman kullanılmaz	Bir aralıktaki tüm sayılar (kalbur daha hızlı); çok büyük sayılar (olasılıksal testler, ör. Miller–Rabin)

Eratosthenes kalburu: p = 2 turu

p = 2: 2·2 = 4'ten başla · yeni 24, yeniden yazılan 0

1	**2**	3	**4**	5	**6**	7	**8**	9	**10**
11	**12**	13	**14**	15	**16**	17	**18**	19	**20**
21	**22**	23	**24**	25	**26**	27	**28**	29	**30**
31	**32**	33	**34**	35	**36**	37	**38**	39	**40**
41	**42**	43	**44**	45	**46**	47	**48**	49	**50**

bu turda yeni işaretlenen

zaten işaretliydi, yeniden yazılır

önceki turlarda işaretlenmiş (bileşik)

asal olduğu kesinleşen

kalın lacivert: p
kesikli turuncu: p·p

2 asaldır; katları 4'ten (2 · 2) başlayarak işaretlenir: 24 yeni işaret

KALBUR · TAHMİN ET

p = 3 turunda (N = 50; iç döngü 3 · 3 = 9’dan başlar) kaç sayı ilk kez işaretlenir?

A 15 (3’ün bütün katları)

B 14

C 7

D 0

p = 3 turu: yeni ve yeniden yazılan işaretler



Yeni 7 işaret (sarı), 7 yeniden yazma (taralı); 6 hiç ziyaret edilmez

Sözde kod: Eratosthenes kalburu

```
1  algoritma KALBUR(N)
2      bileşik[0..N] ← YANLIŞ
3      i ← 2
4      i · i ≤ N iken tekrarla
5          eğer bileşik[i] = YANLIŞ ise
6              j ← i · i
7              j ≤ N iken tekrarla
8                  bileşik[j] ← DOĞRU;  j ← j + i
9          i ← i + 1
10     döndür { k : 2 ≤ k ≤ N, bileşik[k] = YANLIŞ }
```

- **Değişmez:** i. turun başında en küçük asal böleni i'den küçük olan her bileşik işaretli.
- **Analiz:** p'nin turu ≈ N/p işaretleme; Σ 1/p ≈ log log N ⇒ **Θ(N log log N)**.
- **Bellek:** N + 1 bayt, **Θ(N)**: zaman–bellek ödünleşimi.

KOD

C: kalbur (işaretleme)

H02_09_kalbur.c · satır 31-40 — C

```c
    /* i asalsa katlarını i*i'den başlayarak işaretle: daha küçük
     * katlar, daha küçük bir asal tarafından zaten işaretlendi */
    for (i = 2; i * i <= n; i++) {
        if (!bilesik[i]) {
            for (j = i * i; j <= n; j += i) {
                bilesik[j] = 1;
                (*isaretleme)++;
            }
        }
    }
```

ÇIKTI (ÜÇ DİLDE AYNI)

```
         N     π(N)   deneme        bölme  işaretleme  işar./N  bellek(B)
        10        4        4            8           5    0.500         11
       100       25       25          236         104    1.040        101
      1000      168      168         5288        1411    1.411       1001
     10000     1229     1229       117527       16981    1.698      10001
    100000     9592     9592      2745694      193078    1.931     100001
```

ALGORİTMA KİMLİK KARTI

Eratosthenes kalburu *sieve of Eratosthenes*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Ön hesaplama; zaman–bellek ödünleşimi	N önceden bilinir; N + 1 bayt belleğe sığar	—	—	O(N log log N)	O(N)

Özellikler	Katlar p·p'den başlar, dış döngü i·i ≤ N'de durur. N = 10^5'te 193 078 işaretleme (deneme bölmesi: 2 745 694 bölme). N = 10^8 için bayt dizisi ≈ 100 MB, bit dizisi ≈ 12,5 MB.
Değişmez / doğruluk	i. turun başında, en küçük asal böleni i'den küçük olan her bileşik sayı işaretlidir; bu yüzden işaretsiz i asaldır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`sympy.sieve` üçüncü taraf)
Ne zaman kullanılır	Bir aralıktaki tüm asallar, çok sayıda asallık sorgusu (N ≈ 10^7–10^8)
Ne zaman kullanılmaz	Tek bir büyük sayının asallığı; N + 1 bayt belleğe sığmıyorsa (parçalı kalbur gerekir)

Üç dilde: kalburun işaretleme döngüsü

H02_09_kalbur.c · satır 33–40 — C

```c
33      for (i = 2; i * i <= n; i++) {
34          if (!bilesik[i]) {
35              for (j = i * i; j <= n; j += i) {
36                  bilesik[j] = 1;
37                  (*isaretleme)++;
38              }
39          }
40      }
```

H02_09_kalbur.cpp · satır 31–38 — C++

```cpp
31      for (i = 2; i * i <= n; i++) {
32          if (!bilesik[i]) {
33              for (j = i * i; j <= n; j += i) {
34                  bilesik[j] = 1;
35                  isaretleme++;
36              }
37          }
38      }
```

H02_09_kalbur.py · satır 22–28 — Python

```python
22      i = 2
23      while i * i <= n:
24          if not bilesik[i]:
25              for j in range(i * i, n + 1, i):
26                  bilesik[j] = 1
27                  isaretleme += 1
28          i += 1
```

Python'da `range(i * i, n + 1, i)` adımı ve `bytearray`; C++'ta `std::vector<unsigned char>` (vector<bool> değil); C'de işaretçiyle sayaç

hardal satır = C'den farklılaşan satır

HIZLI ÜS · TAHMİN ET

b = 10^{18} için yinelemeli `us_mod(a, b, m)` kaç çarpma yapar? (Saf yöntem: b çarpma)

A 10^{18}

B 60

C 120

D 84

Kare al ve çarp: 3^{13} mod 1000

Kare al ve çarp: 3^{13} mod 1000

$13 = 1101_2$

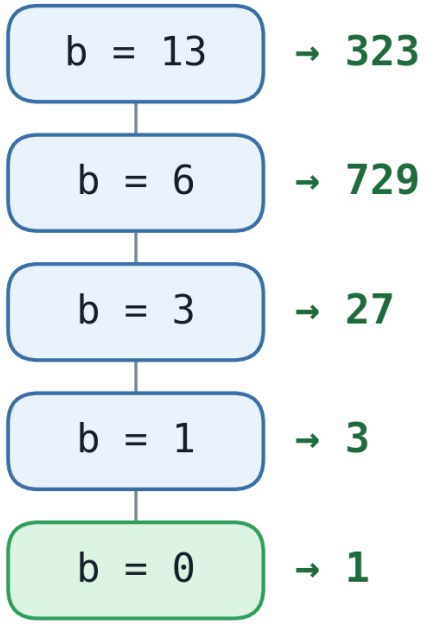
1	1	0	1
2^3	2^2	2^1	2^0

bitler sağdan okunur (b % 2, b /= 2)

tur	b	bit	sonuç	taban
0	13	-	1	3
1	6	1	**3**	**9**
2	3	0	3	**81**
3	1	1	**243**	**561**
4	0	1	**323**	**721**

sarı: o turda değişen · 4 tur, 4 + 3 = 7 çarpma

özyinelemeli: us(3, b)



$13 = 1101_2$: bitler sağdan okunur; bit 1 ise sonuç tabanla çarpılır, her turda taban karesi alınır (sağda özyinelemeli çağrılar)

Sözde kod: yinelemeli kare al ve çarp

```
1  algoritma ÜS-MOD(a, b, m)          ▷ 1 ≤ m < 2³²
2      sonuç ← 1 mod m;  taban ← a mod m
3      b > 0 iken tekrarla
4          eğer b tek ise sonuç ← sonuç · taban mod m
5          taban ← taban · taban mod m
6          b ← ⌊b / 2⌋
7      döndür sonuç
```

- **Değişmez:** her turun başında sonuç · taban^b ≡ a^(b_o) (mod m).
- **Analiz:** tur = bit uzunluğu; çarpma ≤ 2⌊$\log_2 b$⌋ + 2: **O(log b)**.
- **Bellek:** yinelemeli O(1); özyinelemeli O(log b) çağrı yığını.
- **Ön koşul:** $m < 2^{32} \Rightarrow (m-1)^2 < 2^{64}$: çarpım taşmaz.

C: us_mod (gövde)

H02_11_hizli_us.c · satır 29–41 — C

```c
    uint64_t sonuc = 1 % m;
    a %= m;
    *carpma = 0;
    while (b > 0) {
        if (b % 2 == 1) {   /* son bit 1 */
            sonuc = sonuc * a % m;
            (*carpma)++;
        }
        a = a * a % m;      /* kare al */
        (*carpma)++;
        b /= 2;
    }
    return sonuc;
```

ÇIKTI (ÜÇ DİLDE AYNI; SEÇİLİ SATIRLAR)

```
         a                    b           m       sonuç  yin   öz      saf  kâhin
         3                   13        1000         323    7    7       13  =
 123456789              1000000  1000000007   471040903   27   27  1000000  =
         3  1000000000000000000  1000000007   246336683   84   84        -  -
```

ALGORİTMA KİMLİK KARTI

Hızlı üs alma ($a^b \bmod m$) *modular exponentiation by squaring*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Böl ve fethet: kare al ve çarp	$b \geq 0$; $1 \leq m < 2^{32}$: $(m-1)^2 < 2^{64}$	O(log b)	O(log b)	O(log b)	O(1) / O(log b)

Özellikler	Çarpma sayısı = b'nin bit uzunluğu + 1 bitlerinin sayısı $\leq 2\lfloor \log_2 b \rfloor + 2$ ($b = 10^{18}$ için 84; saf yöntem 10^{18}). Özyinelemeli sürüm log b derinlikte çağrı yığını kullanır. RSA ve Fermat testinin temel işlemi.
Değişmez / doğruluk	Yinelemeli sürümde her turun başında: sonuc · taban^b ≡ a^(başlangıçtaki b) (mod m)
Kütüphane karşılığı	C: yok (`pow()` `double` döndürür, mod almaz) · C++: yok · Python: `pow(a, b, m)`
Ne zaman kullanılır	Büyük üsler ($b \approx 10^{18}$), modüler aritmetik, kriptografi, matris üssüyle yineleme bağıntıları
Ne zaman kullanılmaz	Küçük sabit üsler (`x * x * x` daha açık); $m \geq 2^{32}$ iken taşmasız çarpma yoksa

Üç dilde: kare al ve çarp

H02_11_hizli_us.c · satır 32–42 — C

```c
    while (b > 0) {
        if (b % 2 == 1) {   /* son bit 1 */
            sonuc = sonuc * a % m;
            (*carpma)++;
        }
        a = a * a % m;      /* kare al */
        (*carpma)++;
        b /= 2;
    }
    return sonuc;
}
```

H02_11_hizli_us.cpp · satır 34–44 — C++

```cpp
    while (b > 0) {
        if (b % 2 == 1) {   // son bit 1
            sonuc = sonuc * a % m;
            carpma++;
        }
        a = a * a % m;      // kare al
        carpma++;
        b /= 2;
    }
    return sonuc;
}
```

H02_11_hizli_us.py · satır 25–32 — Python

```python
    while b > 0:
        if b % 2 == 1:      # son bit 1
            sonuc = sonuc * a % m
            carpma += 1
        a = a * a % m       # kare al
        carpma += 1
        b //= 2
    return sonuc, carpma
```

C ve C++'ta çarpım 64 bitte sarar ($m < 2^{32}$ ön koşulu); Python'da `int` sınırsızdır, `b //= 2` tamsayı bölmesidir

hardal satır = C'den farklılaşan satır

Öklid'in en kötü durumu: ardışık Fibonacci sayıları

Öklid adım merdiveni

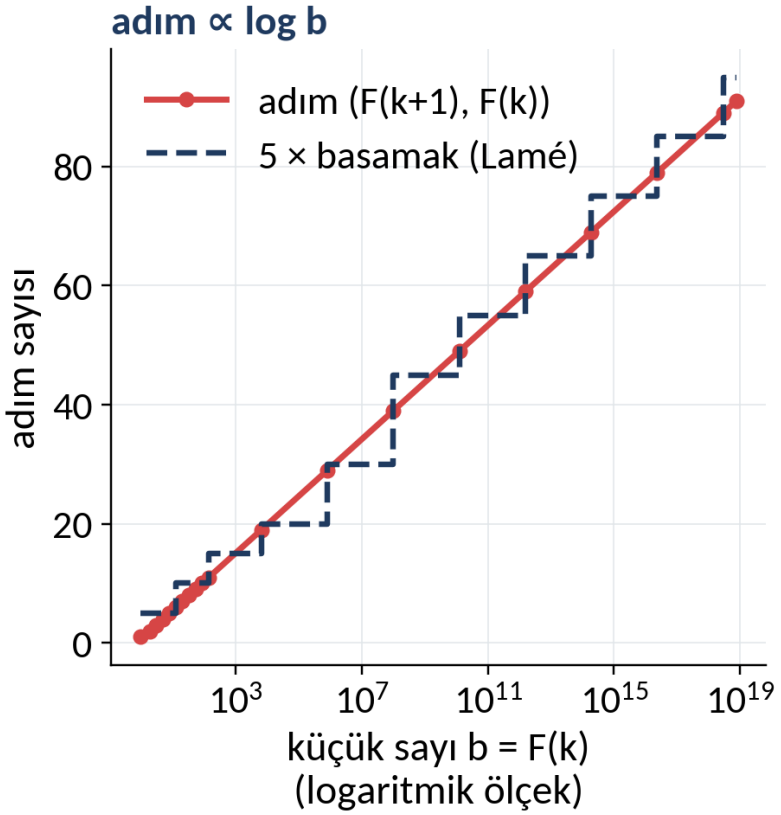
ebob(1071, 462): 3 adım

```
1071 = 2·462 + 147
462 = 3·147 + 21
147 = 7·21 + 0
```

Fibonacci çiftinde her bölüm q = 1: kalan en yavaş biçimde küçülür → en kötü durum (Lamé)

ebob(89, 55): 9 adım

```
89 = 1·55 + 34
55 = 1·34 + 21
34 = 1·21 + 13
21 = 1·13 + 8
13 = 1·8 + 5
8 = 1·5 + 3
5 = 1·3 + 2
3 = 1·2 + 1
2 = 2·1 + 0
```



ebob(a, b) = ebob(b, a mod b): adım sayısı O(log min(a, b)); en kötü durum ardışık Fibonacci çiftleri (Lamé: adım ≤ 5 × basamak sayısı)

En büyük alt dizi toplamı: Kadane’ye giriş

en iyi aralık [2..2] = 59

	0	1	2	3	4	5	6	7	8	9
	31	-41	**59**	26	-53	58	97	-93	-23	84
uzat	—	-10	49							
simdiki	31	-10	59							
en_iyi	**31**	31	**59**							
karar	B	U	Y							

U: uzat · Y: yeniden başla · sarı: en_iyi güncellendi

i = 2’de uzat = −10 + 59 = 49 < 59: a[2]’den yeniden başla (Y); en_iyi = 59

KADANE · TAHMİN ET

i = 6 adımında (a[6] = 97) şimdiki ve en_iyi ne olur? (i = 5'te şimdiki = 90, en_iyi = 90)

A 97 ve 97

B 187 ve 187

C 90 ve 187

D 187 ve 155

i = 6: en iyi aralık [2..6] = 187

en iyi aralık [2..6] = 187

	0	1	2	3	4	5	6	7	8	9
	31	-41	59	26	-53	58	**97**	-93	-23	84
uzat	—	-10	49	85	32	90	187			
simdiki	31	-10	59	85	32	90	187			
en_iyi	**31**	31	**59**	**85**	85	**90**	**187**			
karar	B	U	Y	U	U	U	U			

U: uzat · Y: yeniden başla · sarı: en_iyi güncellendi

Sarı: en_iyi güncellendi; U uzat, Y yeniden başla. Sonraki adımlar (−93, −23, 84) en_iyi'yi değiştirmez

Sözde kod: Kadane

```
1  algoritma KADANE(a[0..n-1])      ▷ n ≥ 1
2      şimdiki ← a[0];  en_iyi ← a[0]
3      i ← 1'den n-1'e kadar her i için
4          uzat ← şimdiki + a[i]
5          eğer uzat < a[i] ise şimdiki ← a[i]
6          değilse şimdiki ← uzat
7          eğer şimdiki > en_iyi ise en_iyi ← şimdiki
8      döndür en_iyi
```

- **Toplama sayıları:** kübik $n(n + 1)(n + 2)/6$, karesel $n(n + 1)/2$, Kadane $n - 1$.
- **$n = 10^5$:** 166 671 666 700 000 · 5 000 050 000 · 99 999.
- **Kadane:** her durumda $\Theta(n)$, ek bellek $O(1)$.

KOD

C: kadane (döngü)

H02_13_en_buyuk_alt_dizi.c · satır 76–90

C

```c
    for (size_t i = 1; i < n; i++) {
        long long uzat = simdiki + a[i];
        s.topl++;
        if (uzat < a[i]) {
            simdiki = a[i];
            s_bas = i;
        } else {
            simdiki = uzat;
        }
        if (simdiki > s.toplam) {
            s.toplam = simdiki;
            s.bas = s_bas;
            s.son = i;
        }
    }
```

ÇIKTI (ÜÇ DİLDE AYNI)

```
Bentley (n=10): kübik 187 [2..6] · karesel 187 [2..6] · Kadane 187 [2..6]
Tümü negatif (n=3): kübik -1 [1..1] · karesel -1 [1..1] · Kadane -1 [1..1]
Tek eleman (n=1): kübik 5 [0..0] · karesel 5 [0..0] · Kadane 5 [0..0]
```

ALGORİTMA KİMLİK KARTI

Kadane (en büyük alt dizi toplamı) *Kadane's algorithm, maximum subarray sum*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Tek geçiş, artımlı güncelleme (H13: DP)	n ≥ 1; alt dizi boş olamaz	Θ(n)	Θ(n)	Θ(n)	O(1)

Özellikler	n − 1 toplama; aralığı (bas, son) da verir; tamamı negatif dizide en büyük elemanı döndürür. $O(n^3)$ ve $O(n^2)$ çözümlerle aynı sonucu verir (Bentley, 2000).
Değişmez / doğruluk	i. adımdan sonra `simdiki`, a[i]'de biten en büyük alt dizi toplamıdır; `en_iyi`, a[0..i] içindeki en büyük alt dizi toplamıdır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Tek geçişte en iyi bitişik aralık (ör. en kârlı dönem); akış verisi; 2B en büyük alt matris probleminin iç adımı
Ne zaman kullanılmaz	Bitişik olmayan seçimler (altküme problemi); boş alt diziye izin veren sözleşmede başlangıç değerleri 0 olmalıdır

Üç dilde: aynı döngü, farklı sonuç (Örnek 12, Δ)

H02_12_buyuk_tamsayi.c · satır 20–27 — C

```c
uint64_t iki_us(int n)
{
    uint64_t x = 1;
    for (int i = 0; i < n; i++) {
        x = x * 2;
    }
    return x;
}
```

H02_12_buyuk_tamsayi.cpp · satır 21–28 — C++

```cpp
std::uint64_t iki_us(int n)
{
    std::uint64_t x = 1;
    for (int i = 0; i < n; i++) {
        x = x * 2;
    }
    return x;
}
```

H02_12_buyuk_tamsayi.py · satır 17–21 — Python

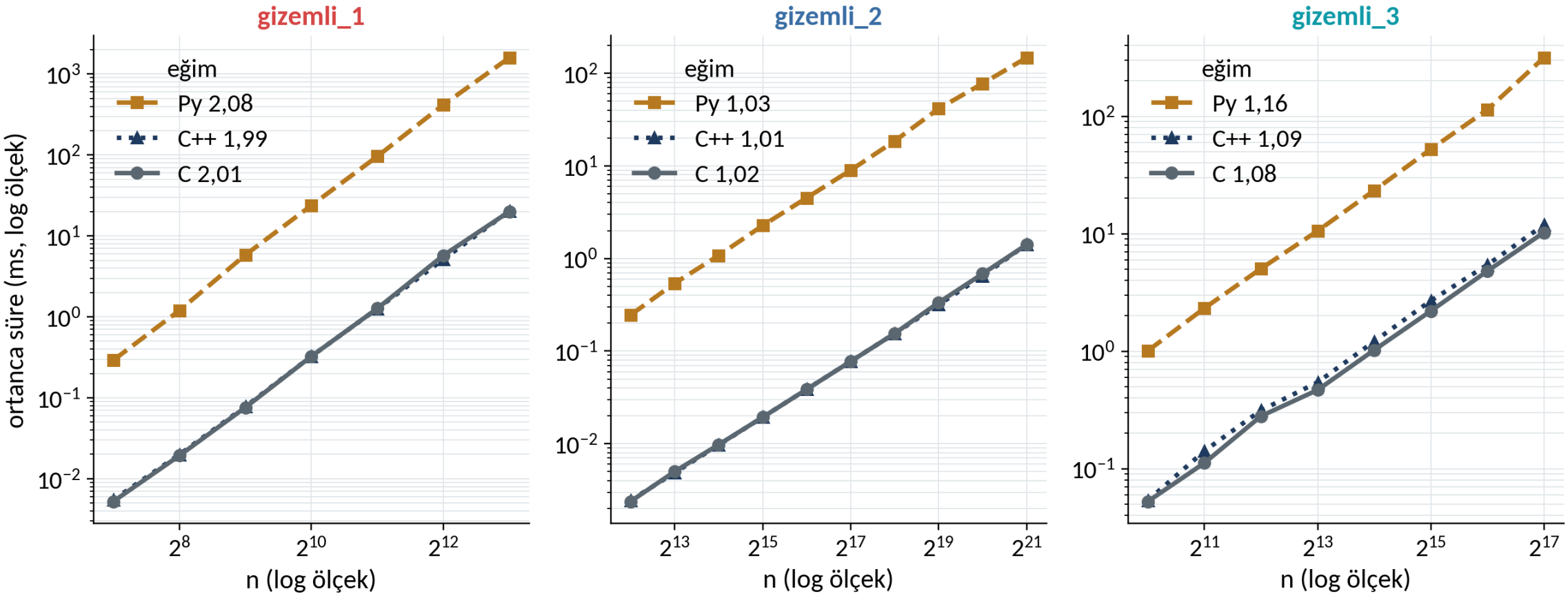
```python
def iki_us(n):
    x = 1
    for i in range(n):
        x = x * 2
    return x
```

C ve C++'ta `uint64_t` 2^{64} modunda sarar (n ≥ 64 için 0, tanımlı davranış); Python `int`'i kesin değeri tutar, bedeli süredir

hardal satır = C'den farklılaşan satır

Haftanın Deneyi: üç kara kutuyu ölçümle sınıflandır

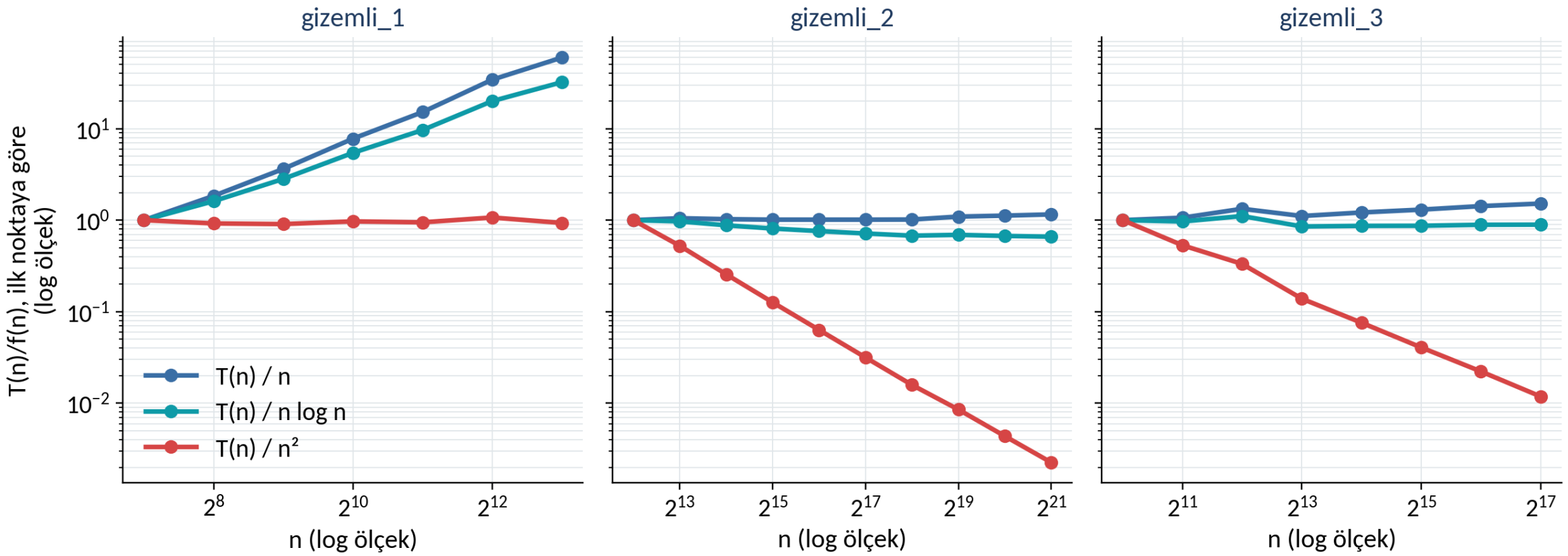
Log–log grafik: eğim (üs) dilden bağımsız, sabit çarpan değil



Log–log eğimler (C -O2): 2,01 · 1,02 · 1,08; doğrular dillerde paralel, dikey kayma sabit çarpan (bu makinede)

n mi, n log n mi? T(n)/f(n) düzlük testi

Hangi f(n) ile bölünce düz kalıyor? (C -O2, ortak ızgara)



Doğru f için T(n)/f(n) düz kalır. Sınırlılık: C -O0'da gizemli_2 yanlışlıkla n log n sınıflandı (ilk oranlar 2,36 ve 2,23)

Hata Avcısı: koşulda gizli maliyet

Hatalı: her turda strlen

H02_HA3_strlen_kosulda.c · satır 28–33 — C

```c
28      for (size_t i = 0; i < strlen(s); i++) {    /* HATA: her turda O(n) */
29          if (s[i] == ',') {
30              s[i] = ';';
31              degisen++;
32          }
33      }
```

Düzeltilmiş: uzunluk bir kez

H02_HA3_strlen_kosulda_duzeltilmis.c · satır 27–33 — C

```c
27      size_t uzunluk = strlen(s);                 /* DÜZELTME: bir kez */
28      for (size_t i = 0; i < uzunluk; i++) {
29          if (s[i] == ',') {
30              s[i] = ';';
31              degisen++;
32          }
33      }
```

n = 160 000 karakterde 284,0 ms ↔ 0,208 ms; katlama oranları ≈ 4 ↔ ≈ 2 (bu makinede). Derleyici uyarısı: 0.

Hata Avcısı: sekiz hata, hiçbiri derleyici uyarısı üretmez

Hata	Belirti (bu makinede)	Çözüm
HA1 “O(n) her zaman daha hızlı”	“O(n)” 546,6 ms; melez 1,8 ms	Sabitleri ölç; küçük n’de kaba kuvvet
HA2 en iyi durumla ölçüp genellemek	aranan = a[0]: oran ≈ 1,00 → “Θ(1)”	Durumları ayrı ölç ve raporla
HA3 koşulda strlen	oranlar ≈ 4	Uzunluğu bir kez hesapla
HA4 Python x in liste	n = 32 000’de 3,9 s	Görülenleri set’te tut
HA5 kalburda int taşması	N = 100 000’de SIGSEGV; UBSan: signed overflow	i <= n / i, long long sayaç
HA6 j = 2i, dış döngü n’ye kadar	$N = 10^6$: 2 775 210 ↔ 2 122 048 işaretleme	j = i * i
HA7 a * b % m 64 biti aşar	$m \approx 10^{12}$’de Fermat testi 0 (doğrusu 1)	$m < 2^{32}$ denetimi ya da carp_mod
HA8 ölü kodu ölçmek	-O2’de süre ≈ 0 ms → “O(1)?!”	Sonucu kullan; önce ölçümden şüphelen

Dil Penceresi: aynı görünen kod, farklı maliyet

Öğe	C	C++	Python
Dizgi uzunluğu	`strlen(s)`: O(n)	`s.size()`: O(1)	`len(s)`: O(1)
Üyelik sınaması	elle tarama O(n)	`std::unordered_set` ort. O(1)	`x in kume` ort. O(1); `x in liste` O(n)
Sabit genişlikli tamsayı	`uint64_t` sarar; işaretli taşma tanımsız	C ile aynı	`int` sınırsız; maliyet basamakla büyür
Üs alma	`pow()`: double, 53 bit	`std::pow`: double	`pow(a, b, m)` kesin ve modüler
Kayan nokta taşması	`pow(2.0, 10000.0)` → inf	→ inf	`2.0 ** 10000` → OverflowError
Türkçe harfle dolgu	`%-5s` bayt sayar	`setw` bayt sayar	`f"{s:<5}"` karakter sayar

Mühendis Gözüyle ve Merak Kutusu

Karmaşıklık bütçesi

Saniyede ≈ 10^8 işlem, 1 s bütçe: $n \le 10^4 \rightarrow n^2$; $n \le 4{,}5 \cdot 10^6 \rightarrow n \log n$; daha büyük $n \rightarrow n$.

Gizli karesel kod

Döngü içindeki her çağrının maliyetini sorun: `strlen`, `x in liste`, `list.index`, başa ekleme.

Sayaç + süre

Süre gürültülüdür; işlem sayacı kuramla birebir karşılaştırılır. Ön koşulları kodda yazılı tutun.

Merak: deneme bölmesi polinom mu?

Girdi boyutu bit sayısı b ise √n ≈ 2^(b/2): üstel. 128 bitlik bir asal için ≈ $1{,}8 \cdot 10^{19}$ bölme.

MİNİ-OBS 2 PROJESİ

Mini-OBS 2 v2.1: işlem sayaçları ve yinelenen numara tespiti

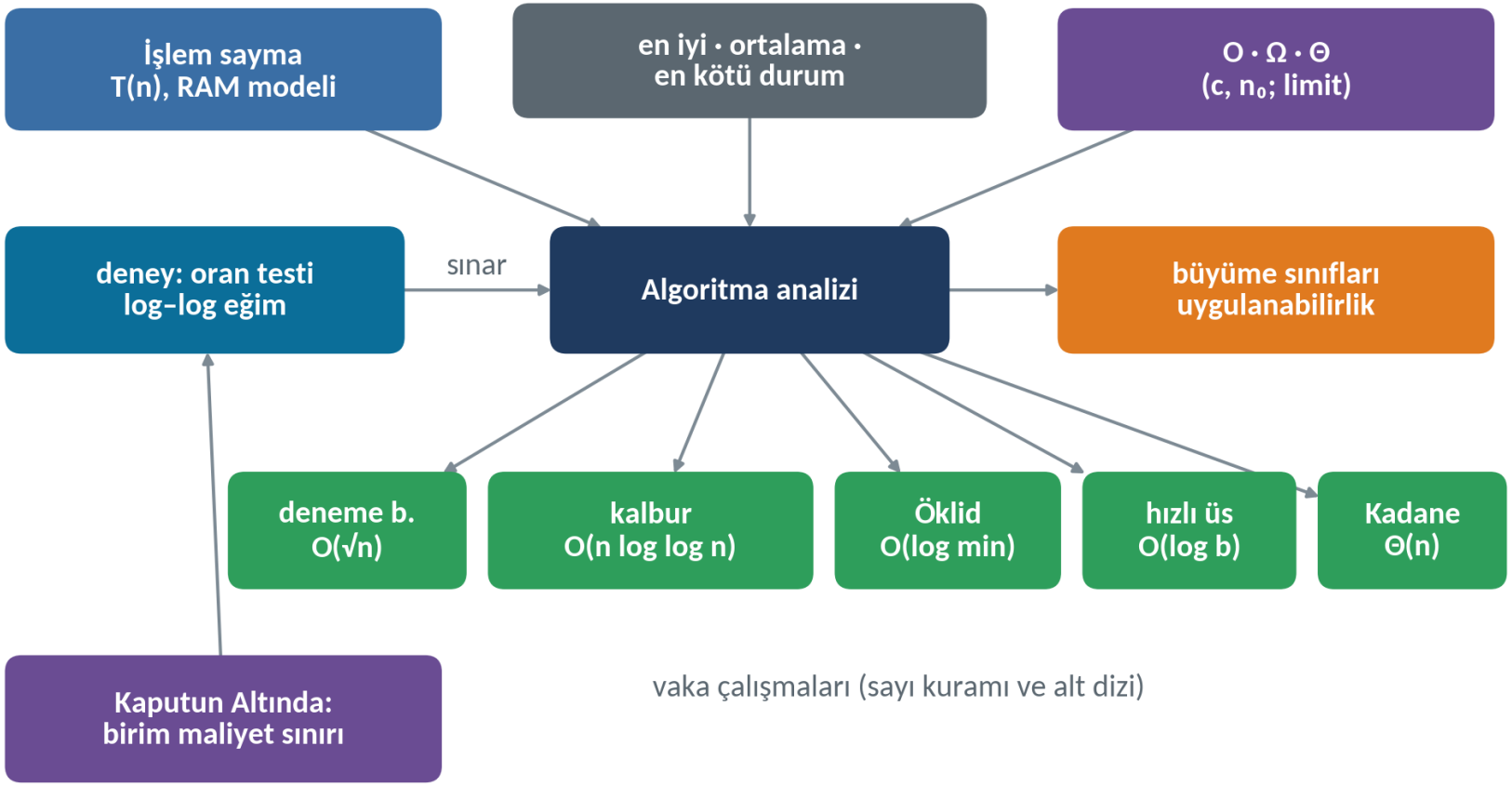
- **Sayaçlar:** `sayac.h`'deki `Sayac` yapısı ve `SAYAC_EKLE`; her fonksiyonun başına T(n)'yi yorum olarak yazın.
- **yinelenen.c:** `yinelenen_var_mi` (erken çıkış) ve `yinelenen_bul` (tüm çiftler, tam $n(n-1)/2$ karşılaştırma).
- **Kabul:** `./denetle testler/yinelenen_k3.csv` → 435 karşılaştırma, 3 çift; `--erken` → 185.
- **Oran testi:** $n \geq 4\,000$ için her katlama oranı 3,5–4,5; en az 7 katlamada log–log eğim 1,9–2,1.
- **Kestirim:** 10^6 kayıtta ≈ 340 s (≈ 5,7 dk): H6'da sıralayarak O(n log n), H10'da karma tablosuyla ortalama O(n).

İLERİ BAKIŞ

Bu haftanın konusu nerede derinleşiyor?

- Asimptotik analiz: Algoritma Analizi dersinde amortize analiz, olasılıksal analiz ve alt sınır kanıtlarıyla derinleşir.
- Kadane, H13'te dinamik programlamanın ilk örneği olarak yeniden yorumlanır.
- Hızlı üs alma H6'da böl ve fethet, kriptografi derslerinde ise RSA'nın temel işlemi olarak yeniden karşımıza çıkar.

Özet: Hafta 2 kavram haritası



Analizin üç aracı (işlem sayma, durumlar, O–Ω–Θ) ölçümle (oran testi, eğim) sınanır; dört algoritma vaka çalışmasıdır

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `for (i = n; i > 0; i /= 3) s++;` kaç kez döner? Sınıfı Θ ile yazın.

2 Bir ölçümde T(2n)/T(n) ≈ 4 çıktı: sınıf nedir? ≈ 2 çıksaydı hangi iki sınıf olabilirdi ve nasıl ayırırdınız?

3 Kalbur deneme bölmesinden neden hızlıdır, bedeli nedir? N = 10^8 için kalbur dizisi ne kadar bellek ister?

Hazırlık: Hafta 3 — Özyineleme ve Geri İzleme

- **Okuyun:** ders notunun Bölüm 4.3'ünde ve Ek A'da özyinelemeli `us_mod_oz`; çağrı yığınının $\lfloor \log_2 b \rfloor + 2$ derinliğe nasıl ulaştığını izleyin.
- **Tekrar:** AP-1 H9 (özyineleme) ve H4 (bit operatörleri): H3'te alt kümeleri bit maskesiyle üreteceğiz.
- **Uygulayın:** atölyedeki oran testi deneyini kendi makinenizde tamamlayın; Mini-OBS 2 v2.1 ölçüm raporunu hazırlayın.
- **Atölyeler (ders notu 11.2–11.5):** 1 "Karmaşıklığını söyle" kartları (12 kart) · 2 oran testi: üç gizemli fonksiyon · 3 Kadane yarışması · 4 asal sayma: $\pi(10^7) = 664\,579$
- **Düşünün:** sekiz vezir bulmacasında ≈ 4,4 milyar yerleşimi tek tek denemek 1 saniyelik bütçeye sığar mı (uygulanabilirlik tablosu)?